==========

SATQUERY AI - COMPREHENSIVE PROJECT DOCUMENTATION & TECHNICAL DOSSIER

Smart India Hackathon (SIH 2026) | Problem Statement: SIH26167 (ISRO / SAC)

==========

## 1. EXECUTIVE SUMMARY & PROBLEM STATEMENT

Problem Statement ID: SIH26167 | Theme: Space Technology (Software) | Organization: ISRO

Core Challenge: Remote sensing imagery from optical (Cartosat, Sentinel-2) and Synthetic Aperture Radar (SAR, RISAT, Sentinel-1) sensors provides essential Earth observation data. However, existing AI tools operate in silos, requiring specialized GIS expertise.

Why Generic VLMs Fail: Standard vision-language models (e.g. GPT-4V/BLIP) cannot handle multispectral reflectance bands, SAR backscatter textures, complex spatial resolutions, or temporal change pairs.

Solution Goal: SatQuery AI provides an intelligent, natural-language vision assistant that autonomously interprets queries, plans workflows, coordinates specialist models, and returns evidence-grounded answers with full execution traces.

## 2. PROPOSED SOLUTION & ARCHITECTURAL HIGHLIGHTS

- End-to-End Orchestration: LangGraph StateGraph DAG with conditional routing and session persistence.

- Specialist Model Registry: Modular specialist models for Single-Image VQA, Captioning, Grounding, Bi-temporal Change Detection, and Optical + SAR Cross-Modal Fusion.

- Multi-Modal Sensor Alignment: Unified co-registration validation between optical and SAR modalities.

- Dual-Mode Inference: Model-backed inference with deterministic spectral/backscatter fallback for resilient offline execution.

- Rich Evidence Generation: Visual bounding boxes, change difference maps, confidence scoring, and automated PDF audit report generation.

- Interactive Web Dashboard: Built on React + Leaflet for seamless image upload, geospatial overlay visualization, execution trace inspection, and report download.

3. IMPLEMENTATION & CODEBASE MODULE BREAKDOWN

A. LangGraph Agent Layer (backend/agent/)

- state.py: AgentState schema tracking user inputs, validation flags, task routes, specialist outputs, and thread_id.
- graph.py: LangGraph StateGraph DAG with validate_inputs, classify_intent, specialist execution nodes, and fuse_evidence.
- task_classifier.py: Intent classification router interpreting natural language text and image counts.
- tool_registry.py: Extensible registry managing specialist model instances.

B. Specialist Model Implementations (backend/models/)

- base.py: BaseSpecialistModel abstract class defining execution interfaces and performance timers.
- vqa/model.py: RemoteSensingVQAModel integrating BLIP VQA with spectral heuristic fallback.
- captioning/model.py: RemoteSensingCaptionModel for dense natural language scene descriptions.
- grounding/model.py: RemoteSensingGroundingModel for query-guided bounding box localization.
- change_detection/model.py: ChangeDetectionModel performing pixel-difference mapping and thresholding.
- change_vqa/model.py: ChangeVQAModel answering natural language comparative questions across two dates.
- optical_sar/model.py: OpticalSARFusionModel combining optical spectral indices with SAR backscatter.

C. Preprocessing, Validation & Evidence (backend/)

- validation/validator.py: InputValidator verifying formats, dimensions, channels, and corruption checks.
- preprocessing/registration.py: ImageRegistration verifying spatial alignment and co-registration.
- evidence/generator.py & report.py: Evidence generator for mask overlays and PDF audit reports.
- evaluation/metrics.py: Implementation of Accuracy, Binary IoU, and Binary F1 calculation formulas.

D. API Gateway & Frontend (backend/api/ & frontend/)

- api/endpoints/agent.py: Unified /api/v1/agent route executing the StateGraph with thread session persistence.
- frontend/src/main.jsx: React + Leaflet web dashboard with interactive map viewer and trace monitor.

4. TECHNICAL METHODOLOGY & EVALUATION STRATEGY

- Parameter-Efficient Domain Adaptation: Utilizes LoRA (Low-Rank Adaptation) on BIFOLD BigEarthNet v2.0 and RSVQA to adapt vision-language representations to Sentinel-1 (SAR) and Sentinel-2 (optical) modalities without catastrophic forgetting.
- Cross-Modal Sensor Alignment: Leverages physical complementarity — Optical imagery provides spectral surface reflectance (vegetation/water), while SAR radar provides cloud-penetrating structural texture and roughness.
- Observable Auditable Trace: Produces structured execution traces with timestamps, selected models, and parameter configurations without leaking opaque internal chain-of-thought.
- Verification & Testing: 100% test coverage across all phases (tests/verify_agent.py, verify_vqa.py, verify_phase2.py, verify_phase3.py, verify_phase4.py, verify_evaluation.py).

5. 3-MINUTE SIH PITCH & DEMONSTRATION SCRIPT

Minute 1 (The Problem & Vision): 'Honorable evaluators, satellite imagery is crucial for national disaster response, urban planning, and environmental surveillance. Yet non-experts struggle with complex GIS tools, and generic LLMs hallucinate on remote-sensing data. We present SatQuery AI — an agentic multimodal vision-language assistant designed specifically for remote sensing.'

Minute 2 (Live Demonstration & Routing): 'Let us demonstrate: When we upload a Sentinel-2 image and ask "Highlight the water body", our LangGraph StateGraph agent validates input integrity, routes the query to our Grounding Specialist, and displays precise bounding boxes on our Leaflet viewer. When given dual-date images and asked "What changed between dates?", it seamlessly activates our Bi-Temporal Change model to generate a difference heatmap and textual explanation.'

Minute 3 (Cross-Modal Fusion & Evidence): 'For complex queries combining Optical and SAR data, SatQuery AI extracts complementary spectral and backscatter features. Every result includes confidence scores, an auditable execution trace, and a downloadable PDF evidence report. SatQuery AI bridges the gap between raw satellite data and actionable intelligence.'